

Exception Handling

Overview

- Exception handling is an important ingredient of a robust application and should be included in the application design of every .NET program.
- Exception handling helps applications identify and respond to exceptional events in a known and robust manner.
- This enhances the correctness of an application, which naturally improves customer satisfaction.

- Unfortunately, exception handling is often an afterthought, which leads to poorly implemented solutions for exception handling and a less robust application.
- Starting with application design, exception handling should be included in all aspects of application planning and development

What is an exception?

- Exceptions are exceptional events or error conditions within an application and are categorized as system or application exceptions.
- System exceptions are raised by the Common Language Runtime (CLR) and include null reference, out of memory, divide by zero, and stack overflow exceptions.
- Application exceptions, considered custom exceptions, are thrown by the application, not by the CLR.

- Some exceptional events or error conditions are detectable by application logic, not by the runtime.
- Application exceptions are useful in those circumstances.
- As an example, constructors that fail are not always detectable by the CLR.
- In addition, constructors implicitly return void, which prevents returning an error code.
- For these reasons, throwing an application exception in the failed constructor is the best solution.

- Exceptions should be used only for exceptions.
- Exception handling is not a substitute for transfer of control or a *goto* statement.
- Exceptions are expensive when compared with conventional transfer of control.
- Whenever possible, applications should preempt exceptions and avoid the accompanying costs.

- Performing data validation, where incorrect values can cause exceptions, is one measure that prevents exceptions.
- Data validation and, if necessary, returning an error code is undoubtedly cheaper than raising an exception.

- The overuse of exceptions makes code harder to read and maintain.
- Errors that frequently occur should not be handled with exception handling, but with error codes or returns.
- Remember, exceptions should be reserved for exceptional events.
-

Exception Example

- A common exception is *dividing by zero*.
- Exceptions are generally linked to an action.
- For dividing by zero, the action is a divisor of zero.
- Integer division, where the divisor is zero, triggers a divide by zero exception.
- (However, floating-point division by zero does not cause an exception; instead, infinity is returned.)

- The following code (Next slide) causes an unhandled divide by zero exception, which terminates the application:

```
using System;
Namespace Exception
{
    public class Starter
    {
        public static void Main()
        {
            int var1=5, var2=0; var1/=var2; // exception occurs
        }
    }
}
```

- Place code that is likely to raise an exception in a *try* block because code in a *try* block is protected from exceptions.
- Exceptions raised in the *try* block are trapped.
- The stack is then walked by the CLR, searching for the appropriate exception handler.

- Code not residing in a *try* block is unprotected from exceptions.
- In this circumstance, the exception eventually evolves into an unhandled exception.
- As demonstrated in the previous code, an unhandled exception is apt to abort an application.

- In the following code (Next slide) , the divide by zero exception is caught in a *try* block.
- Trapping, catching, and handling an exception are separate tasks.
- The *catch* statement consists of a *catch* filter and block.
- The *DivideByZeroException* filter catches the divide by zero exception.
- The *catch* block handles the exception, which is to display the stack trace.
- The proximity of the infraction is included in the stack trace.
- Execution then continues at the first statement after the *catch* block.

```
using System;
Namespace Exception
{
    public class Starter{
    public static void Main(){
        try {
            int var1=5, var2=0; var1/=var2; // exception occurs
        }
        catch(DivideByZeroException except) {
            Console.WriteLine("Exception "+except.StackTrace);
        } } } }
```

Standard Exception Model

- .NET offers a common exception model.
- Exception handling is implemented within the CLR and is not specific to a managed language.
- Each managed language exposes exception handling using language-specific syntax.
- The uniform exception model contributes to language independence.
- In addition, exceptions raised in one managed language can be caught in a different managed language.

- Prior to .NET, there were competing models for error handling:
 - Microsoft Visual Basic 6.0, Win32 SDK, MFC, COM, and more.
- In a component-driven architecture, this disparity contributed to complexity and potential product instability.
- In .NET, exception handling between disparate components is consistent, simple, and stable.

- As part of the common exception model, .NET advocates structured exception handling, which is essentially the *try* and *catch* statements.
- C-based and Java developers are probably familiar with structured exception handling.

Structured Exception Handling

- A structured exception evaluates the stack to determine when code is protected and where an exception is caught and handled.
- When exceptions occur, the stack is walked in search of an exception handler.
- When a matching *catch* filter is located, the exception is handled in the exception handler, which is the *catch* block.
- This sets the scope of the exception on the stack.
- After the exception is handled, the CLR unwinds the stack to that scope

Try Statement

- *Try* blocks are observers; they watch for exceptions in protected code.
- Place code prone to raising exceptions in a *try* block.
- Do not attempt to protect an entire application in a *try* block—there are more convenient and practical means of accomplishing the same feature.

- As mentioned, exceptions are stack-based.
- The following code (Next slide) contains a fence-post error that happens when the bounds of the array are exceeded.
- The result is an exception in the unprotected *MethodA* method.
- *Main* calls *MethodA*, where the scope of *Main* encompasses *MethodA*.
- For this reason, the *try* block in *Main* extends protection to *MethodA*.
- The exception is trapped in *Main*.

```
using System;
Namespace Exception{
public class Starter{
public static void Main(){
try {
MethodA();
}
catch(Exception except) {
Console.WriteLine(except.Message);
} }
public static void MethodA() {
int [] values={1,2,3,4};
for(int count=0; count<=values.Length; ++count) {
Console.WriteLine(values[count]);
}
} } }
```

- The syntax of the *try* statement:

```
try { protected } catch1(filter1) { handler1 } catchn(filtern) { handlern } finally {terminationhandler}
```

- *Try* statements must be paired with either a *catch* statement or a *finally* statement.
- There can be zero or more *catch* statements attached to a *try* statement; there are zero or one *finally* statements.
- If both *catch* and *finally* are present, the *catch* statement should precede the *finally* statement.

The following code has various combinations of *try*, *catch*, and *finally* statements:

```
// try..catch
```

```
try {
```

```
}
```

```
catch(Exception e) {
```

```
}
```

```
// try..finally
```

```
try {
```

```
}
```

```
finally{
```

```
}
```

```
// try..catch..finally  
try {  
}  
catch(Exception e) {  
}  
finally{  
}
```

```
// try..catches..finally  
try {  
    }  
    catch(Exception e) {  
    }  
    catch {  
    }  
finally{  
}
```

Catch Statement

- *Catch* statements filter and handle exceptions.
- When an exception is raised, the filter sets which exceptions are handled at that scope in the stack.
- If the filter matches the exception, the exception is suppressed, and control is transferred to the adjoining *catch* block to be handled.
- The CLR searches the stack for an appropriate filter.
- If a matching filter is not found, the exception becomes an unhandled exception.

- Exceptions must be derived from *System.Exception*.
- The *catch* filter defines an exception type and catches exception of that type or any descendants.
- The exception object contains details of the exception, such as a user-friendly message describing the exception.
- The exception object caught in the filter is accessible only in the *catch* block.
- *System.Exception* is the base class of .NET exceptions and the generic filter.

- A *System.Exception* filter catches all managed exceptions.
- Derived classes of *System.Exception* catch more-specific exceptions.
- In the previous code, the *DivideByZeroException* filter caught integer divide by zero exceptions and nothing else.
- A *try* block can be appended to multiple catches for catching distinct exceptions.
- The catches are ordered from specific to generic

The following code has several exception filters, from specific to generic:

```
using System;
Namespace Exception{
    public class Starter{
    public static void Main(){
    try {
    int var1=5, var2=0; var1/=var2;
    }
    catch(DivideByZeroException except) {
    Console.WriteLine("Exception "+except.Message); }
    catch(System.ArithmeticException except) {
        Console.WriteLine("Exception "+except.Message); }
    catch(Exception except) {
        Console.WriteLine("Exception "+except.Message);
    } } } }
```

- In the preceding code, *DivideByZeroException* is very specific and catches only divide by zero exceptions.
- *ArithmeticException* is less specific and catches a variety of arithmetic exceptions, including the divide by zero exception.
- *Exception* catches all managed exceptions, which includes divide by zero and arithmetic exceptions.
- In the preceding code, the exception is caught at the *DivideByZeroException* catch handler.

- The *catch* filter is optional, and the default is *catch all*.
- Although *System.Exception* catches any managed exceptions, *catch all* catches both managed and unmanaged exceptions.
- In most circumstances, native exceptions are mapped to managed exceptions, which are thrown in the *RaiseTheException* native function.
- *RaiseTheException* uses the *RaiseException* API to construct the managed exception.
- Exceptions not mapping to managed exceptions are outside this normal mechanism.
- A catch with no *catch* filter is a *catch all*.

This code has a *catch all*.

```
using System;
Namespace Exception{
public class Starter{
public static void Main(){
try {
    int var1=5, var2=0; var1/=var2; }
catch(DivideByZeroException except) {
    Console.WriteLine("Exception "+except.StackTrace); }
catch {
    // catch remaining managed and unmanaged exceptions
}}}}}
```

Propagating Exceptions

- Exceptions are not always handled locally where the exception is caught.
- It is sometimes beneficial to catch an exception and then propagate the exception.
- *Propagating* an exception is catching and then rethrowing the exception.
- Rethrowing an exception continues the search along the call stack to find an appropriate handler.

Here are some reasons to propagate an exception:

- There is a centralized handler for the exception.
- There are several reasons for implementing centralized handlers, including code reuse.
- Instead of handling an exception in various locations in an application, concentrate code for certain exceptions in a centralized handler.
- Wherever the exception is raised, the proper response is to record the exception and then delegate to the centralized handler.
- A central handler can be used to handle all exceptions in a single place.

- Resources required to handle the exception are not available locally.
- For example, an exception is raised because of an invalid database connection.
- However, the correct connection string is read from a file not available where the exception occurs.
- The solution is to propagate the exception to a handler that has access to the file resource.

- Propagate unwanted exceptions caught in the umbrella of the exception filter.
- This would be useful for catching all *DataException* types with the exception of the *DuplicateNameException*.
- One solution would be to write 12 individual catches—one for each of the data exceptions except for the *DuplicateNameException* exception.
- A better solution is to catch the *DataException* type and propagate the *DuplicateNameException* when necessary.
- This is one *catch* statement versus 12 *catch* statements and eliminates redundant code.

- Catch an exception to gather information or to report on an exception, and then propagate the exception.
- To propagate an exception, rethrow the same exception or another exception in the *catch* block.
- An empty *throw* statement propagates the caught exception.
- Alternatively, throw a different exception.

Finally Statement

- The *finally* block is the termination handler.
- When an exception is raised, protected code after the infraction is not executed.
- What if it is cleanup code? If cleanup code is not executed, resources are left dangling.
- Termination handlers are the solution.

- Code that must execute, regardless of an exception occurring, is placed in a termination handler.
- When an exception is raised, the *finally* blocks within scope are called before the stack is unwound.

- Note that the termination handler is executed even when there is no occurrence of an exception.
- Execution simply falls through the *try* block into the attached *finally* block.
- In the termination handler, you could close a file, release a database connection, or otherwise manage resources.

Here is a typical termination handler:

```
using System;
using System.IO;
Namespace Exception{
public class FileWriter{
public static void Main(){
StreamWriter sw=null;
try {
sw=new StreamWriter("date.txt");
sw.Write(DateTime.Now.ToLongTimeString());
throw new ApplicationException("exception");
// dangling code
}
finally {
sw.Close();
Console.WriteLine("file closed");
} } } }
```

Exception Information Table

- The CLR refers to the Exception Information Table to track protected code in an efficient manner.
- Because of the Exception Information Table, there is no overhead associated with an exception unless an exception occurs.

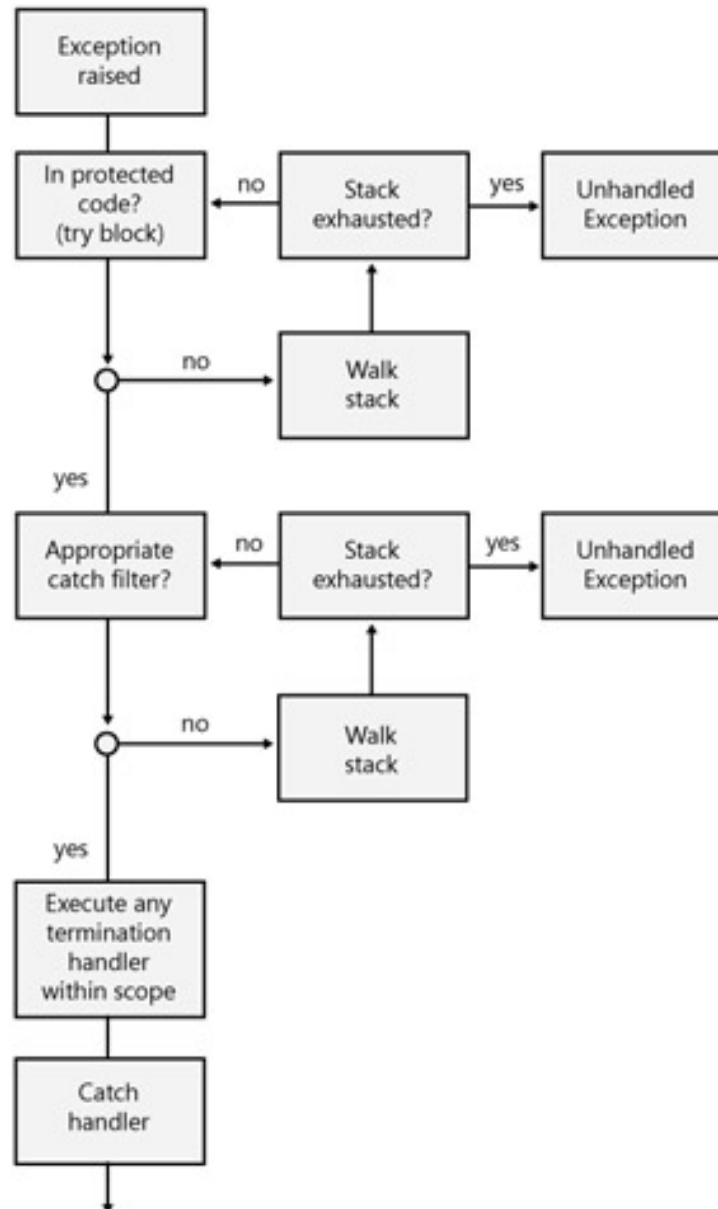
- An Exception Information Table is constructed for every managed application.
- The table has an entry for each method in the program.
- Each entry is an array, where the array elements describe the filters and handlers of that method.
- Entries in the array represent a *catch* filter, *user-filtered* handler, *catch* handler, or *termination* handler.
- User-filtered handlers use the *when* clause and are available in Visual Basic .NET, but not in C#.

- When an exception occurs, the CLR consults the Exception Information Table.
- The entry for the method hosting the exception is searched for a filter that matches the exception.
- If the array is empty or a matching filter is not found, the entry of the outer method is examined next.
- When the boundary of the application is reached, the exception is considered unhandled

Nested Try Blocks

- *Try* blocks can be nested.
- The order of evaluation is more complex with nested *try* blocks.
- This is the order of execution when an exception occurs:
 - The first step is to find an appropriate *try* block.
 - If an exception is raised outside a protected block, the exception is not trapped and is therefore unhandled.
 - From the *try* block, walk the stack until a matching *catch* filter is found.
 - If a matching filter is not found, the exception is unhandled.
 - This defines the scope of the exception.
 - Before the stack is unwound, *finally* blocks within the scope of the exception are run.
 - The innermost *finally* blocks are executed first.
 - The *catch* block of the filter is executed as the exception handler.
 - Execution continues at the first statement after the *catch* block.
 - *Finally* blocks at scope of exception are executed.

The sequence when an exception is raised in a nested *try* block.



Code with several nested *try* blocks

```
using System;
Namespace Exception{
public class Starter{
public static void Main(){
try {
    Console.WriteLine("outer - try");
    try {
        Console.WriteLine("inner - try");
        throw new ApplicationException("exception");
    }
    finally {
        Console.WriteLine("inner - finally");
    }
}
catch(Exception except){
    Console.WriteLine("outer - catch");
}
finally {
    Console.WriteLine("outer - finally");
}
}
}
}
```

System.Exception

- *System.Exception* is the base exception class.
- All exceptions in .NET are derived from *System.Exception*.
- *System.SystemException* is the base class for system exceptions raised by the CLR, such as *System.Data.DataException* or *System.FormatException*.
- *SystemException* is derived directly from *System.Exception*.

System.Exception Functions

- *System.Exception* has four constructors:
- *public Exception*¹()
- *public Exception*²(string message)
- *public Exception*³(string message, Exception innerException)
- *protected Exception*⁴(Serialization info, StreamingContext context)

- *Exception*¹ is the default constructor.
- *Exception*² constructor sets the user-friendly message of the exception.
- *Exception*³ constructor also sets the inner exception, which is the originating exception.
- *Exception*⁴ deserializes an exception raised remotely.

- The *Exception* class has several other helpful functions.
- Table (Next Slide) lists the important methods of the class.

Method Name	Result
<i>GetBaseException</i>	Returns the root exception in a chain of exception objects
<i>GetObjectData</i>	Serializes data of the <i>Exception</i> class
<i>GetType</i>	Returns the type of the exception
<i>ToString</i>	Concatenates the name of the exception object with the user-friendly message

System.Exception Properties

Property	Description	Type	Read/Write
<i>Data</i>	Returns a dictionary collection that provides optional user-defined details of exception.	<i>IDictionary</i>	R
<i>HelpLink</i>	Link to a help file for the exception.	<i>string</i>	R/W
<i>HResult</i>	The HRESULT, which is a 32-bit error code common to COM, assigned to the exception. This is a protected property.	<i>int</i>	R/W

<i>InnerException</i>	When exceptions are propagated, the inner exception represents the previous exception.	<i>Exception</i>	R
<i>Message</i>	User-friendly message describing the exception.	<i>string</i>	R
<i>Source</i>	Name of application or object where exception occurred.	<i>string</i>	R/W
<i>StackTrace</i>	String representation of the call stack when the exception occurred.	<i>string</i>	R
<i>TargetSite</i>	Reference to the method where exception is raised.	<i>MethodBase</i>	R

Application Exceptions

- Application exceptions are custom exceptions and are thrown by the application, not by the CLR.
- Application exceptions are derived from *System.ApplicationException* or *System.Exception*.
- *System.ApplicationException* adds nothing to *System.Exception*.

- A custom exception derived from *System.Exception* accomplishes the same feat.
- When several custom exceptions are planned, create a custom base exception class to categorize the exceptions.
- For convenience and maintainability, deploy application exceptions together in a separate assembly.

- Do not create an application exception for an existing exception.
- Research the available system exceptions to avoid replicating an existing exception.

Steps for creating an application exception:

- Name the application exception.
 - As a best practice, the class name should have the *Exception* suffix, as in *ApplicationException*.
- Derive the application exception from *System.Exception*.
- Define constructors that initialize the state of the application exception.
 - This includes initializing members inherited from the base class.
- Within the application exception, refine *System.Exception* as desired, such as by adding attributes that further delineate this specific exception.

- To raise an application exception, use the *throw* statement.
- You can also throw system exceptions.
- Thrown exceptions are considered software exceptions.
- The CLR treats software exceptions as standard exceptions.

- *throw* syntax:
 *throw exceptioninstance*¹;
 *throw*²;
- The second syntax is specialized:
 - It is available in the *catch* block, but nowhere else.
 - This version of the *throw* statement rethrows as an exception caught in the *catch* block.

- Application exceptions are typically prompted by an exceptional event.
- What is an exceptional event?
- A strict definition does not exist.
- You define the basis of the event using whatever criteria are appropriate.

- Remember, raising an exception simply for transfer of control or a nonexceptional event is bad policy.
- In an application, the following could be considered exceptional events where throwing an application exception is warranted:
 - Constructor fails to initialize the state of an object.
 - A property does not pass validation.
 - Null parameters.
 - An exceptional value is returned from a function.

- *ConstructorException* is an application exception.
- Throw this exception when a constructor fails.
- It refines the *System.Exception* base class with name of the type and time of exception

Exception Translation

- In some circumstances, the CLR catches an exception and rethrows a different exception.
- The inner exception of the translated exception contains the original exception.
- Invoking a method dynamically using reflection is one such circumstance.
- Exceptions raised in methods invoked by *MethodInfo.Invoke* are automatically trapped and converted to *TargetInvocationException*.

Remote Exceptions

- Exceptions sometime occur in remote code.
- An exception that is raised in a different application domain is a remote exception.
- Remote exceptions include exceptions thrown in a .NET Remoting application or a Web service application.

Unhandled Exceptions

- Unhandled exceptions are not handled directly in application code; they are handled in a global handler.
- The global handler reports the exception in an error box that offers the Debug, Send Error Report, and Don't Send buttons.
- For applications within the realm of a debugger, a potential unhandled exception is handled in the debugger.

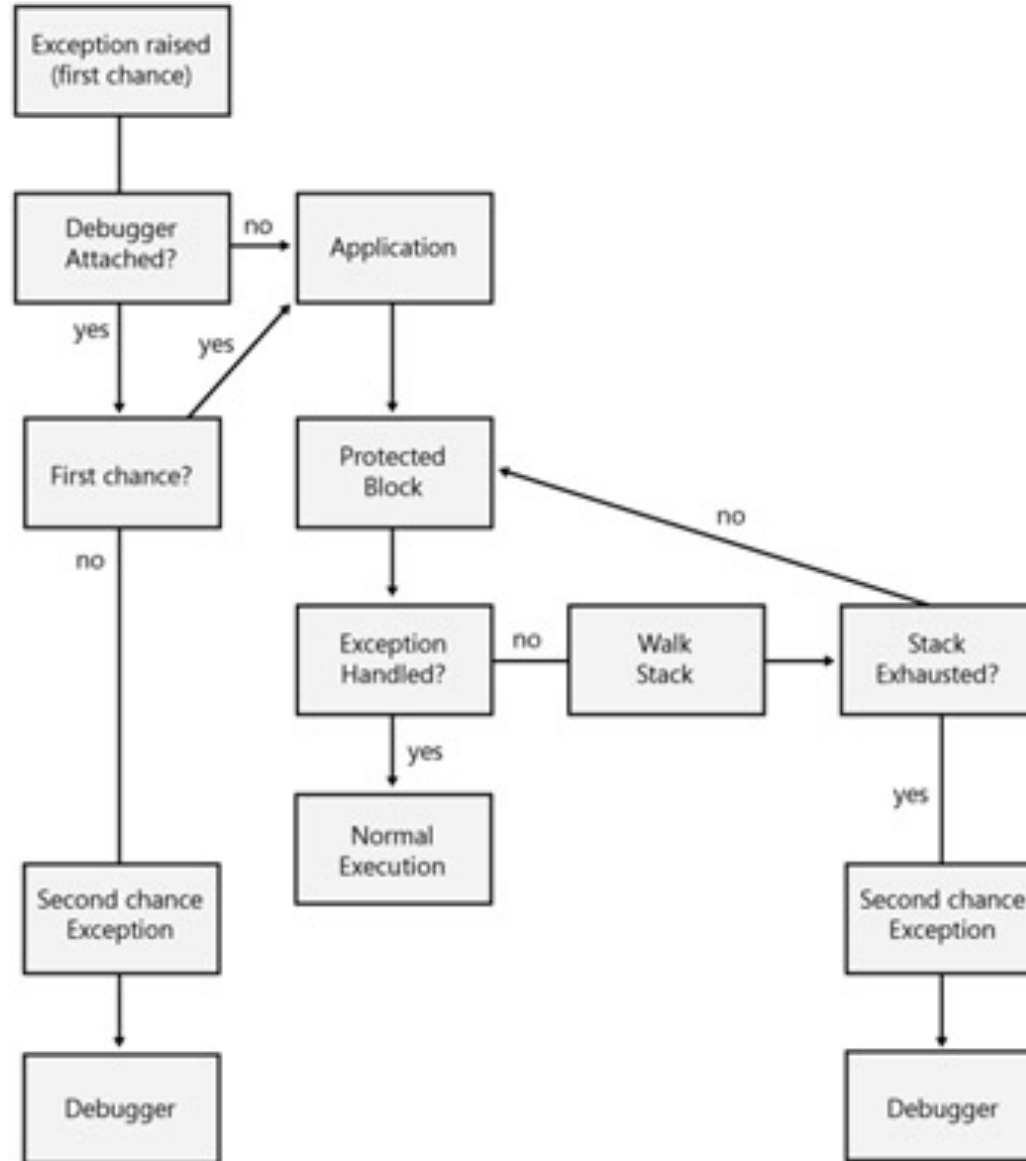
Life cycle of an exception?

- Exceptions are initially categorized as first chance exceptions.
- If the application is attached to a debugger, the debugger is first consulted about the exception.
- Debuggers typically ignore first chance exceptions, and the exception is forwarded to the application next.
- When no debugger is present, the first chance exception is immediately sent to the application.

- If the application does not handle the first chance exception, the exception becomes a high-priority second chance exception.
- If a debugger is attached, the second chance exception is handled by the debugger.
- Upon finding a second chance exception, the Visual Studio debugger transfers the user to the location in the source code where the exception happened.

- If no debugger is present, execution is transferred to a global exception handler, which displays an error dialog box and then terminates the application.

The life cycle of an exception



- Applications can trap unhandled exceptions.
- The mechanism is different for Windows Forms and for Console applications.
- For Windows Forms, add a handler to the *Application.ThreadException* event.
- For Console applications, the handler is added to the *AppDomain.UnhandledException* event.

- Do not use the unhandled exception handler to catch all exceptions.
- Proper application design identifies specific exceptions that an application should anticipate.
- Those exceptions should be caught and handled within the confines of structured exception handling.
- Reserve the unhandled exception method for unanticipated exceptions.

Managing Exceptions in Visual Studio

- Visual Studio 2008 can configure exceptions for debugging.
- The Exception Assistant provides helpful information to developers when an unhandled exception is raised.
- The Exceptions dialog box alters how the Visual Studio debugger handles exceptions.

Exception Assistant

- The Exception Assistant appears in Visual Studio when an unhandled exception occurs.
- It displays a translucent frame that partially obscures the application code.
- The source code that prompted the exception is highlighted and tethered to the Exception Assistant window.

The Exception Assistant



- The Exception Assistant header identifies the unhandled exception and offers a brief explanation.
- The Troubleshooting Tips section offers hints to diagnose the exception.
- Each hint is also a link to further information.
- The Actions pane specifies two actions:
 - New Detail displays the properties of the exception object.
 - Copy Exceptional Detail To The Clipboard copies basic information on the exception onto the Clipboard.
 - The Exception Assistant can be disabled or otherwise configured from the Tools menu.
 - Select the Options menu item.
 - Configure the Exception Assistant in the Debugging pane.

Exceptions Dialog Box

- In the Exceptions dialog box, the Visual Studio debugger can be instructed to interrupt on first chance exceptions.
- Open the Exceptions dialog box from the Debug menu.
- Choose the Exceptions menu item.

The Exceptions dialog box



- The Thrown and User-unhandled columns have a series of option boxes that are organized into categories and specific exceptions.
- Visual Studio debugger interrupts on first chance exceptions for exceptions selected in the Thrown column.
- For protected code, the debugger intercedes before the exception is caught.
- In the development phase of an application, it can be instructive to be notified of exceptions that would otherwise be consumed in an exception handler.
- The second column selects specific user-handled exceptions to break on.
- The Add button appends application exceptions to the list of available exceptions that are selectable in the dialog box.
- These exceptions can later be deleted.

Metadata and Reflections

- .NET assemblies adhere to the Portable Executable Common Object File Format (PE COFF).
- PE COFF files have headers and sections, such as a PE and CLR Header.
- .NET assemblies consist of metadata and MSIL (Microsoft Intermediate Language) code.

- Metadata describes the types and other characteristics of the assembly.
- You can inspect the metadata of an assembly using reflection.
- Tools, such as ILDASM (performs various validations), can be used to browse an assembly using reflection.
- Reflection also provides support for late binding.

- ILASM – Intermediate Language Assembler
- ILDASM – Intermediate Language Disassembler

- <http://etutorials.org/Programming/programming+microsoft+visual+c+sharp+2005/Part+III+More+C+Language/Chapter+9+Exception+Handling/Structured+Exception+Handling/>